

RoboGuardian

Governed execution is more resilient than open-loop execution when the world changes.

Self-healing robotic control loop

MuJoCo • bimanual dual SO-101 • CPU inference

github.com/rootz-synapsa/roboguardian



OBSERVE / GOVERN / RECOVER

The world changes. The plan doesn't.

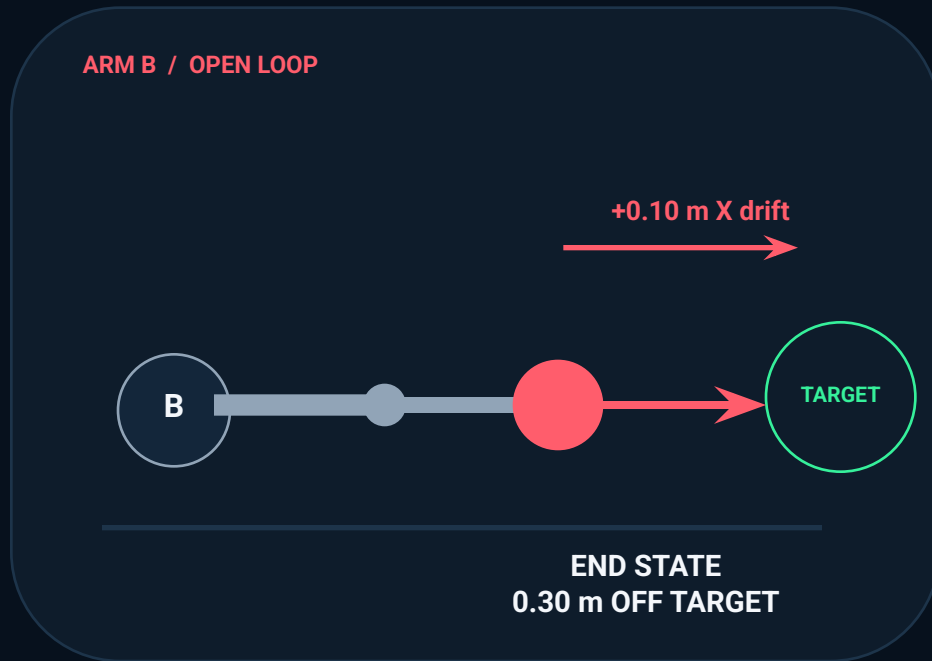
PLAN ONCE

EXECUTE BLIND

100%

failure under disturbance
10 / 10 deterministic trials

results/g3_failure.json



When observation stops, the plan becomes fiction.

Wrap execution in a governance loop



Stale plans never reach the actuators

```
trace = RecoveryTrace(
    stale_target=dict(stale_target)
)

fresh_state = provider.observe(...)
ik = solve_arm_ik(
    goal_pos=fresh_state.observed_pose,
    seed_qpos={...stale_target...}
)

trace.fresh_target = ik["joint_solution_rad"]
_execute_arm(trace.fresh_target, ...)
```

STALE TARGET

Retained for traceability +
solver initialization only



never written
to data.ctl

FRESH TARGET

ACTUATORS

STRUCTURAL INVARIANT `stale_action_executed = False` • G5 + G6

IK seed biases the configuration branch (elbow-up/down) only.
The solve goal always comes from fresh `observe()`. `solve_arm_ik()` uses an internal `MjData` copy, so the live simulation remains untouched until `_execute_arm(fresh_target)`.

Apples-to-apples comparison

ARM A

HAPPY PATH

No disturbance
Open-loop reference
10 trials

ARM B

OPEN LOOP

Object moved mid-run
No recovery
10 trials

ARM C

ROBOGUARDIAN

Object moved mid-run
Governed recovery
10 trials

COMMON PROTOCOL

SEED 42

NOISE = 0

+0.10 m X

@ BEFORE_GRASP

10 TRIALS / ARM

Governance restored 10 / 10 success

CONTROL / NO DISTURBANCE ARM A 10 / 10

ARM B • OPEN LOOP

0 / 10

STALE EXECUTED 10 / 10
FINAL MISS 0.30 m

V
S

ARM C • ROBOGUARDIAN

10 / 10

STALE EXECUTED 0 / 10
RECOVERY 10 / 10

ZERO STALE ACTIONS EXECUTED BY ROBOGUARDIAN

Vision perception without ground truth

MEAN POSITION ERROR

≈ 0.7 cm

WORST-CASE ERROR

2.4 cm

MODEL SIZE

≈ 146K
params

P50 LATENCY, CPU

ONNX Runtime 0.5 ms
OpenVINO 1.2 ms

OPENVINO VS ONNX RUNTIME

0.44×

speedup ratio

≈2.3× slower on this tiny CPU model

NO SPEEDUP CLAIMED

Both median latencies are below 2 ms. Tail latency remains visible in the G7 artifact.

Evidence chain and reproducibility



```
results/  
├ g1_environment.json  
├ g2_baseline.json  
├ g3_failure.json  
├ g4_detection_vision_t03.json  
├ g5_recovery_vision.json  
├ g6_reliability.json  
└ g7_openvino.json
```

G1 PASS

G2 PASS

G3 PASS

G4 PASS

G5 PASS

G6 PASS

G7 PASS

```
python experiments/run_g6_reliability.py \  
    --model models/dual_so101_g3.xml \  
    --trigger before_grasp --delta 0.10 0.0 0.0 --seed 42 --trials 10
```

Scope of validity

SIMULATION FIDELITY

Kinematic carry model;
no physics-based grasping.

DETERMINISTIC EVIDENCE

Fixed seeds, noise = 0;
no confidence interval.

TEST COVERAGE

One disturbance type,
one target position.

DEPLOYMENT BOUNDARY

MuJoCo + CPU only;
no real hardware or certification.

CONTROLLED SIMULATION EVIDENCE $\neq$ FORMAL PROOF

CONCLUSION

RESILIENCE > SPEED

Under the tested scenario, governed execution recovered 10 / 10 while open-loop execution recovered 0 / 10.

01

OBSERVE CHANGE

02

REPLACE STALE PLAN

03

PRESERVE EVIDENCE



SCAN TO CLONE

OPEN SOURCE

CLONE • RUN • INSPECT

github.com/rootz-synapsa/roboguardian